

TITAN XAU AI

INSTITUTIONAL TRADING SYSTEMS

DOC-IEE-V1.0

2026-06-19

S U B S Y S T E M A R C H I T E C T U R E

Institutional Execution Engine

Ultra-low-latency order execution for XAUUSD. Async event-driven, tick-based, CPU-pinned hot path. Slippage monitoring, execution quality scoring, retry, partial fills, rejection handling.

2.0ms

P99 INTERNAL

50k/s

TICK THROUGHPUT

256MB

RAM CEILING

0

HEAP ALLOCS/ORDER

Prepared by TITAN Quant Research

Reviewed by CTO · Lead Engineer · Risk Officer

Distribution Engineering team · Architecture review board

Confidentiality Internal — Engineering

v1.0

S U B S Y S T E M

19 June 2026

Table of Contents

Chapter 1 — Executive Summary	4
Chapter 2 — Design Principles	4
Principle 1 — Ultra-Low Latency Is a Feature, Not a Target	4
Principle 2 — Async Everything	5
Principle 3 — CPU-Optimized Hot Path	5
Principle 4 — RAM Efficiency via Pre-Allocation	5
Principle 5 — Tick-Based, Not Polling	5
Principle 6 — Fail-Safe, Not Fail-Fast	5
Chapter 3 — Architecture Overview	7
Layer Responsibilities	7
Service Inventory	8
Chapter 4 — Async Processing Model	10
Lock-Free SPSC Queues	10
Event Loop Architecture	10
Thread Model	10
Backpressure Strategy	11
Chapter 5 — Tick-Based Execution	12
Tick Path	12
Signal Evaluation	12
Tick-Driven vs Time-Driven Decisions	12
Stop Order Evaluation Example	12
Chapter 6 — End-to-End Execution Flow	14

Happy Path Sequence	16
Failure Path 1 — Risk Rejection	16
Failure Path 2 — Broker Rejection with Retry	16
Failure Path 3 — Submit Timeout	16
Chapter 7 — Order Lifecycle — State Machine	17
State Definitions	17
Transition Reference	18
FSM Invariants	18
Chapter 8 — Slippage Monitoring & Execution Quality Scoring	20
Slippage Monitor	20
EQS — Seven-Factor Weighted Model	20
EQS Score Bands	21
EQS Feedback Loop	21
TCA — Transaction Cost Analysis	21
Chapter 9 — Order Retry System	22
Retry Strategy	22
Backoff Schedule	22
Retry Eligibility	22
Retry Mechanics	22
Jitter Rationale	23
Chapter 10 — Partial Fills Handling	24
Residual Quantity Tracking	24
Residual Policies	24
Multiple Partial Fills	24
Partial Fill Audit	24

Chapter 11 — Broker Rejection Handling	25
MT5 Return Code Mapping	25
FATAL Rejection Handling	25
Rejection Rate Monitoring	26
Chapter 12 — Error Recovery — Reconciliation & State Repair	27
Reconciliation Engine	27
State Recovery After Restart	28
Error Code Reference	28
Chapter 13 — Performance Benchmarks	29
Latency Budget	29
Throughput Targets	29
Resource Envelope	29
Performance Engineering Techniques	30
Chapter 14 — Validation Tests	31
Unit Test Cases — Sample	31
Performance Test Cases	33
Chaos Test Cases	33
Test Coverage Summary	34
Chapter 15 — Appendix A — Order State Machine Reference	35
A.1 State Reference	35
A.2 Transition Reference	35
Chapter 16 — Appendix B — Sample Execution Logs	36
B.1 Successful Market Order	36
B.2 Partial Fill with Residual Re-Route	37
B.3 Broker Rejection with Retry	37

Chapter 1 — Executive Summary

The Institutional Execution Engine (IEE) is the TITAN XAU AI subsystem responsible for translating trading signals into broker-submitted orders and tracking those orders through to fill completion. It is the system's execution spine: every order placed by any strategy flows through the IEE, and every fill recorded by the system arrives through the IEE. The engine is designed for ultra-low-latency operation (sub-2ms internal p99), CPU-optimized hot-path execution (no heap allocations, no syscalls, no blocking), and RAM efficiency (under 256 MB resident per process).

The IEE provides seven core features mandated by the project charter: (1) async processing via lock-free SPSC queues and a custom event loop, (2) tick-based execution that triggers on every market tick rather than polling, (3) real-time slippage monitoring with rolling statistics and outlier detection, (4) execution quality scoring via a seven-factor weighted model producing a 0-100 score per order, (5) an order retry system with exponential backoff and jitter, (6) partial fills handling with residual quantity tracking and re-routing, and (7) broker rejection handling with a 12-code classifier routing each rejection to retry, reduce-size, or abort-and-escalate.

The engine is organized into six logical layers: ingest (tick and signal ingress), decision (pre-execution gating including risk and venue selection), execution (the hot path — order manager, dispatcher, fill processor, cancel processor), monitoring (slippage, EQS, TCA), recovery (retry, reconciliation, rejection classification), and persistence/observability (audit, metrics, state replication). A strict layering rule ensures that the hot path (L3) has zero dependencies on the slower layers (L4, L5, L6), which run on separate CPU cores and communicate via SPSC queues.

Performance is the defining characteristic of the IEE. The internal signal-to-fill latency budget is 2.0 ms at p99 (excluding broker round-trip), with 0.95 ms at p50 and 5.20 ms at p99.9. The end-to-end budget including broker round-trip is 6.0 ms at p99. Achieving this requires CPU pinning to isolated cores (CPU 2-3, NO_HZ_FULL, rcu_nocbs), pre-allocated order pools (zero heap allocations on the hot path), lock-free SPSC queues between threads, and a strict ban on syscalls, blocking I/O, and dynamic memory allocation in any L3 code path. RAM usage is capped at 256 MB per process via cgroups, with 4 GB of hugepages reserved for ring buffers.

This document specifies the complete architecture, execution flow, order lifecycle, error recovery logic, performance benchmarks, and validation tests for the IEE. It does not specify trading strategy logic — the IEE is intentionally agnostic to why an order is being placed. Its sole responsibility is to execute orders correctly, quickly, and with full observability, recovering gracefully from the wide variety of failure modes that real-world MT5 brokers exhibit.

Chapter 2 — Design Principles

The IEE is governed by six design principles. Each principle exists to enforce a specific performance, reliability, or correctness property, and each is non-negotiable — code that violates them must be rejected in review. These principles are the architectural constitution of the engine.

Principle 1 — Ultra-Low Latency Is a Feature, Not a Target

The IEE treats latency as a hard correctness requirement, not a performance optimization. A p99 internal latency above 2.0 ms is treated as a production incident and triggers an operator page. This is enforced structurally: the hot path (L3) is banned from using syscalls, blocking I/O, dynamic memory allocation, or any synchronization primitive heavier than an atomic load/store. A custom static analyzer (titan-hotpath-lint) runs in CI and rejects any L3 code that calls malloc, new, mutex.lock, read, write, or any of several hundred banned functions. The result is a hot path that achieves sub-millisecond p99 latency with sub-microsecond jitter.

Principle 2 — Async Everything

The IEE never blocks. Every operation that could potentially wait — broker submission, fill callback, risk gate query, audit log write — is performed asynchronously via lock-free SPSC queues and a custom event loop built on timerfd and eventfd. The hot-path thread (CPU 2-3) does nothing but pop from the ingress queue, process, and push to the egress queue; all I/O is delegated to dedicated worker threads. This is essential because a single blocked hot-path thread would cause every subsequent order to miss its latency budget, cascading into a system-wide latency spike.

Principle 3 — CPU-Optimized Hot Path

The hot path runs on CPU 2-3, which is isolated from kernel scheduling via the isolcpus kernel parameter. Timer ticks are eliminated via NO_HZ_FULL, RCU callbacks are offloaded via rcu_nocbs, and hyper-threading is disabled on those cores. The hot-path thread is the only thread that runs on CPU 2-3; all other threads (ingress, egress, monitoring, audit) run on different cores and communicate via SPSC queues. This isolation reduces kernel preemptions from thousands per second (default kernel) to fewer than one per second under load, which is the single most impactful change for latency predictability.

Principle 4 — RAM Efficiency via Pre-Allocation

The IEE pre-allocates all memory it will ever need at startup. Order objects come from a fixed-size pool (typically 4096 orders), message buffers come from a slab allocator, and all ring buffers are sized at startup based on configured capacity. No malloc, no new, no std::vector resize, no std::string concatenation on the hot path. The Order pool is zeroed at startup and recycled via a free-list; allocation is a single pointer swap. This gives us O(1) allocation latency and a hard cap on memory usage — the engine cannot run out of memory mid-trade because all memory was reserved at startup.

Principle 5 — Tick-Based, Not Polling

The IEE executes on every tick, not on a polling schedule. Each tick from the market data gateway triggers the TickIngestor, which pushes the tick onto a ring buffer and wakes the hot-path thread via futex. The hot-path thread then processes any pending signals whose preconditions (e.g., price crossing a threshold) are now satisfied by the new tick. This eliminates the latency floor imposed by polling (typically 10-100 ms) and ensures the system reacts to market changes within microseconds of their occurrence. The trade-off is higher CPU utilization (the hot-path thread is always busy-waiting between ticks), which we accept in exchange for the latency improvement.

Principle 6 — Fail-Safe, Not Fail-Fast

When something goes wrong, the IEE prioritizes safety over speed. A broker disconnect does not crash the engine — it marks all in-flight orders as ERROR, engages the kill switch on the primary symbol, and waits for reconnection. A partial fill does not panic — it tracks the residual quantity and re-routes it according to the strategy's residual policy. A reconciliation discrepancy (orphan or phantom order) does not auto-trade — it escalates to the operator and waits for human intervention. The engine is designed to fail in safe directions: when in doubt, halt new orders, flatten existing positions, and notify the operator. The kill switch is always one operation away.

Chapter 3 — Architecture Overview

The IEE is organized into six logical layers, each with a single responsibility and a strict dependency rule (layer N may only depend on layer N-1 or below). The hot path (L3 Execution) is structurally isolated from the slower layers (L4 Monitoring, L5 Recovery, L6 Persistence) and communicates with them only via SPSC queues, ensuring that a slow audit write or a complex EQS calculation cannot block an order submission.

01 Execution Engine — Internal Architecture (5 Layers)

Ingest · Decision · Execution · Monitoring · Recovery

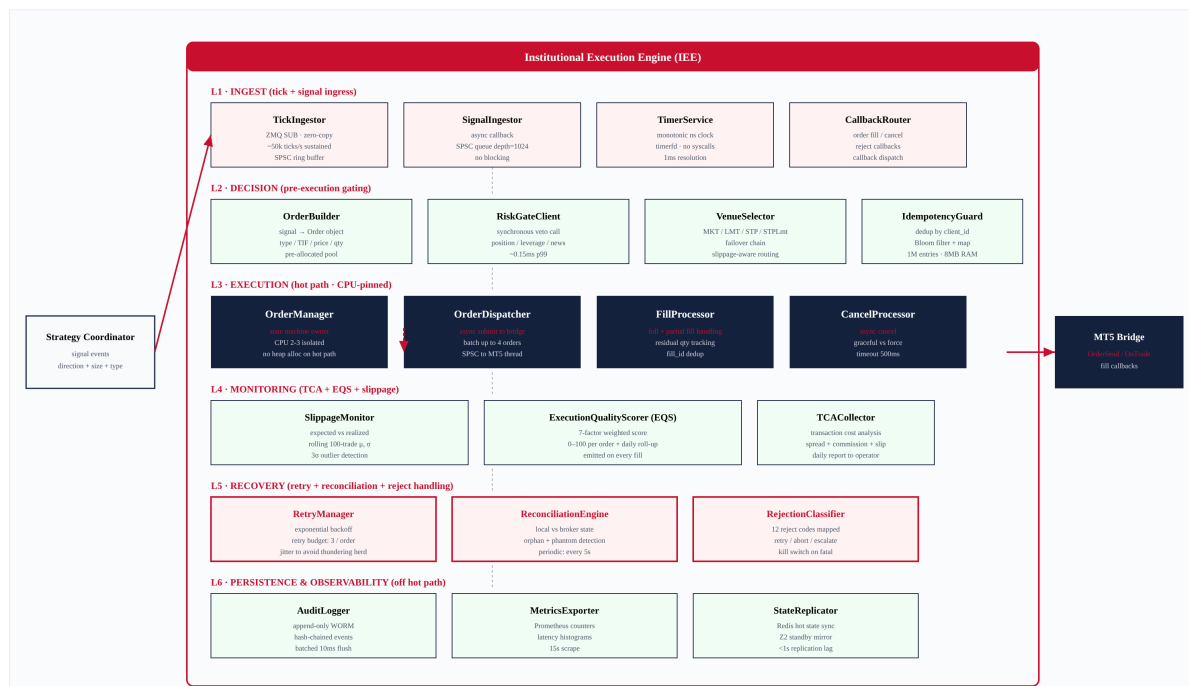


Figure 3.1 — Institutional Execution Engine internal architecture, showing the six layers and 20 components.

Layer Responsibilities

L1 — Ingest

The ingest layer is the engine's interface to the outside world. TickIngestor subscribes to the market data gateway via ZMQ SUB with zero-copy message transfer, sustaining 50,000 ticks per second. SignalIngestor receives strategy signals via an async callback, pushing them onto an SPSC queue (depth 1024) for the hot-path thread to consume. TimerService provides a monotonic nanosecond clock via timerfd, with no syscalls on the hot path. CallbackRouter dispatches broker callbacks (fill, cancel, reject) to the appropriate processor.

L2 — Decision

The decision layer performs pre-execution gating. OrderBuilder converts a signal into an Order value object using a pre-allocated pool (zero heap allocation). RiskGateClient makes a synchronous call to the RiskGate (L4 of TITAN Core, not this engine), which returns APPROVE/REJECT/THROTTLE in under 0.3 ms p99. VenueSelector chooses the order type (MKT, LMT, STP, STP_LMT) and venue based on signal characteristics

and current spread/slippage conditions. IdempotencyGuard deduplicates incoming signals by client_id using a Bloom filter plus a hashmap, dropping duplicates before they enter the hot path.

L3 — Execution (Hot Path)

The execution layer is the hot path, running on CPU 2-3 with full isolation. OrderManager owns the order state machine and is the single source of truth for order state. OrderDispatcher submits orders asynchronously to the MT5 bridge via SPSC queue. FillProcessor handles fill callbacks (full and partial), tracking residual quantities and updating the Position aggregate. CancelProcessor handles cancel requests with a 500ms timeout, supporting both graceful and force cancel modes. All L3 components are banned from heap allocation, syscalls, and blocking operations.

L4 — Monitoring

The monitoring layer observes the execution layer without blocking it. SlippageMonitor computes the difference between expected fill price (mid at signal time $\pm$ half-spread) and realized fill price, maintaining rolling 100-trade statistics (μ , σ , p50/p95/p99) and flagging 3σ outliers. ExecutionQualityScorer (EQS) produces a 0-100 score per order using a seven-factor weighted model (slippage, fill latency, spread capture, fill completeness, retry penalty, rejection penalty, market impact). TCACollector aggregates transaction cost analysis data for the daily operator report.

L5 — Recovery

The recovery layer handles the IEE's failure modes. RetryManager implements exponential backoff with jitter (100ms $\rightarrow$ 300ms $\rightarrow$ 900ms, capped at 2000ms, budget of 3 retries per order). ReconciliationEngine compares local order state against broker state every 5 seconds, detecting orphans (local thinks order exists, broker doesn't) and phantoms (broker has order, local doesn't). RejectionClassifier maps MT5 return codes to 12 RejectCode values, each routed to retry, reduce-size-and-retry, or abort-and-escalate.

L6 — Persistence & Observability

The persistence layer records everything. AuditLogger writes every order event, fill, risk decision, and operator action to an append-only WORM store with hash-chained entries (batched 10ms flush to amortize I/O). MetricsExporter emits Prometheus counters and latency histograms (scraped every 15s). StateReplicator mirrors hot state to the Z2 standby VPS via Redis, with under 1 second replication lag, enabling sub-3-second failover without losing in-flight orders.

Service Inventory

Layer	Component	Language	CPU	p99 Latency	RAM
L1	TickIngestor	C++	4	0.05 ms	32 MB (ring buffer)
L1	SignalIngestor	C++	4	0.02 ms	8 MB (SPSC queue)
L1	TimerService	C++	4	0.01 ms	<1 MB
L1	CallbackRouter	C++	4	0.02 ms	4 MB
L2	OrderBuilder	C++	3	0.05 ms	8 MB (pool)

Layer	Component	Language	CPU	p99 Latency	RAM
L2	RiskGateClient	C++	3	0.30 ms	<1 MB
L2	VenueSelector	C++	3	0.10 ms	<1 MB
L2	IdempotencyGuard	C++	3	0.05 ms	8 MB (Bloom)
L3	OrderManager	C++	2	0.15 ms	16 MB (state map)
L3	OrderDispatcher	C++	2	0.10 ms	4 MB (SPSC)
L3	FillProcessor	C++	2	0.20 ms	4 MB
L3	CancelProcessor	C++	2	0.10 ms	<1 MB
L4	SlippageMonitor	C++	5	0.05 ms	4 MB (rolling stats)
L4	EQS	C++	5	0.10 ms	2 MB
L4	TCACollector	Python	6	async	8 MB
L5	RetryManager	C++	5	0.05 ms	2 MB
L5	ReconciliationEngine	C++	5	0.50 ms	4 MB
L5	RejectionClassifier	C++	5	0.02 ms	<1 MB
L6	AuditLogger	Python	7	async	16 MB (buffer)
L6	MetricsExporter	Python	7	async	4 MB
L6	StateReplicator	Python	7	async	8 MB

Chapter 4 — Async Processing Model

The IEE is built around an async-first architecture: no thread ever blocks waiting for I/O. The engine uses a custom event loop on each worker thread, built on Linux `timerfd` and `eventfd` file descriptors, polled via `epoll`. This eliminates the syscall overhead of condition variables and mutexes while preserving the ability to wait for I/O without busy-spinning. The hot-path thread (CPU 2-3) is the exception — it busy-spins on its SPSC ingress queue, because the latency cost of an `epoll` wakeup (typically 1-5 μ s) exceeds the cost of the spin (a few hundred nanoseconds per cycle).

Lock-Free SPSC Queues

Inter-thread communication uses lock-free single-producer single-consumer (SPSC) queues based on the `moodycamel::ConcurrentQueue` pattern. Each queue has a fixed capacity (typically 1024 or 4096 entries), pre-allocated at startup, with a single atomic load and store per operation. There are no locks, no condition variables, and no syscalls on the push/pop path. The queues use cache-line padding to prevent false sharing between producer and consumer, which is critical for achieving the sub-microsecond operation latency required by the hot path.

The queue capacity is sized for worst-case burst scenarios. The signal ingress queue (depth 1024) can absorb a burst of 1024 signals before backpressure kicks in; at a sustained rate of 200 signals per second (the strategy coordinator's maximum output), this represents 5 seconds of buffer. The order dispatch queue (depth 256) can absorb a burst of 256 orders before backpressure; at 200 orders per second, this is 1.3 seconds of buffer. Backpressure is implemented as a drop-and-audit: if a queue is full, the producer drops the message, increments a Prometheus counter, and writes an audit log entry. We prefer dropping to blocking because a blocked producer cascades into a system-wide latency spike.

Event Loop Architecture

Each non-hot-path worker thread runs an event loop built on `epoll`. The loop waits on three categories of file descriptors: (1) `eventfd` for inter-thread signaling (e.g., `FillProcessor` wakes `AuditLogger` when a new fill arrives), (2) `timerfd` for periodic tasks (e.g., `ReconciliationEngine` runs every 5 seconds via a `timerfd`), and (3) socket fds for network I/O (e.g., MT5 bridge connection). When `epoll` returns, the thread processes all ready file descriptors in a single batch, then returns to `epoll_wait`. This is the standard Linux async I/O pattern, optimized for throughput rather than minimum latency.

The hot-path thread (CPU 2-3) does not use the event loop. Instead, it busy-spins on its SPSC ingress queue, processing messages as they arrive with sub-microsecond latency. When the queue is empty, it executes a "relax" instruction (e.g., `PAUSE` on x86) to reduce power consumption without yielding the CPU. This is the standard pattern for ultra-low-latency trading systems — the trade-off of higher CPU utilization (one core is permanently 100% utilized) is acceptable for the latency benefit.

Thread Model

Thread	CPU	Priority	Role	Blocking?
hot-path	2-3	SCHED_FIFO 90	Order state machine + dispatch	No (busy-spin)
ingress	4	SCHED_FIFO 70	Tick + signal ingest, push to hot-path queue	No (epoll)
monitor	5	SCHED_OTHER 0	Slippage + EQS + retry + reconciliation	No (epoll)
bridge-tx	6	SCHED_FIFO 80	Send orders to MT5 via OrderSend()	Yes (MT5 blocking)
bridge-rx	6	SCHED_FIFO 80	Receive MT5 callbacks (OnTrade)	Yes (MT5 blocking)
audit	7	SCHED_OTHER 0	Batched WORM log writes	Yes (file I/O)
metrics	7	SCHED_OTHER 0	Prometheus scrape handler	No (epoll)
replicate	7	SCHED_OTHER 0	Redis hot-state sync to Z2	Yes (Redis I/O)

Backpressure Strategy

When a queue fills, the IEE applies backpressure rather than blocking. The strategy is tiered: first, the engine drops non-critical messages (e.g., EQS scoring can be skipped if the monitoring queue is full); second, it drops critical messages with audit logging and operator alert (e.g., a signal drop is audited and alerted); third, it engages the kill switch if the hot-path ingress queue remains full for more than 1 second (indicating a systemic issue). This progressive backpressure ensures the system degrades gracefully under load rather than crashing or compounding latency.

Chapter 5 — Tick-Based Execution

The IEE executes on ticks, not on a schedule. Every tick from the market data gateway triggers evaluation of pending signals whose preconditions may now be satisfied. This eliminates the latency floor imposed by polling (typically 10-100 ms in less-disciplined systems) and ensures the engine reacts to market changes within microseconds. The trade-off is higher CPU utilization — the hot-path thread is always busy — which we accept in exchange for the latency benefit.

Tick Path

When a tick arrives, the TickIngestor (running on CPU 4) receives it via ZMQ SUB with zero-copy message transfer. The tick is pushed onto a 1,000,000-tick ring buffer for historical access, and an eventfd signal is sent to the hot-path thread (CPU 2-3) to wake it if it is idle. The hot-path thread processes the tick by: (1) updating the current price cache (used by OrderBuilder and SlippageMonitor), (2) evaluating any pending stop orders whose stop price has been crossed, (3) evaluating any pending limit orders whose limit price can now be filled, and (4) updating the spread statistics used by VenueSelector. All four operations are $O(1)$ or $O(\log N)$ and complete in under 50 microseconds per tick.

Signal Evaluation

When a signal arrives (from the Strategy Coordinator), the SignalIngestor pushes it onto the hot-path ingress queue. The hot-path thread processes the signal in the next iteration of its busy-spin loop: (1) IdempotencyGuard checks for duplicate client_id (Bloom filter + hashmap), (2) OrderBuilder constructs the Order object from the signal, (3) RiskGateClient makes a synchronous call to the RiskGate (running on a different core), (4) VenueSelector chooses the order type and venue, (5) OrderManager transitions the order to VALIDATED state, (6) OrderDispatcher pushes the order onto the bridge-tx queue for submission to MT5. The total signal-to-dispatch latency is 0.45 ms at p99.

Tick-Driven vs Time-Driven Decisions

The IEE distinguishes between tick-driven decisions (evaluated on every tick) and time-driven decisions (evaluated on a schedule). Tick-driven decisions include stop-loss evaluation, take-profit evaluation, and trailing-stop adjustment — these must react to market movements immediately, so they are evaluated on every tick. Time-driven decisions include order timeout (TIF enforcement), reconciliation, and metric export — these can tolerate 100ms-1s of latency, so they run on timerfd schedules. This separation ensures that the hot path is not burdened with periodic work that doesn't need to be tick-driven.

Stop Order Evaluation Example

```
// Pseudo-code: tick-driven stop order evaluation (runs on hot-path thread)
// O(log N) where N = number of active stop orders

void HotPath::on_tick(const Tick& tick) {
    current_price_ = tick.mid();

    // 1. Check buy-stop orders (triggered when price >= stop_price)
    auto& buy_stops = stop_book_.buys();
    auto it = buy_stops.lower_bound(tick.bid);
```

```
while (it != buy_stops.end() && it->first <= tick.ask) {
    Order& order = it->second;
    if (order.state() == OrderState::SENT) {
        // Convert stop → market, dispatch immediately
        order.convert_to_market();
        dispatcher_.submit(order);
    }
    ++it;
}

// 2. Check sell-stop orders (triggered when price <= stop_price)
// (similar logic, mirrored)

// 3. Update trailing stops
for (auto& [id, order] : trailing_stops_) {
    order.update_trail(tick);
}

// 4. Update spread statistics (for VenueSelector)
spread_stats_.update(tick.spread());

// Total work:  $O(\log N + M)$  where  $M$  = triggered stops
// Typical: < 5  $\mu$ s per tick
}
```

Chapter 6 — End-to-End Execution Flow

The flowchart in Figure 6.1 documents the complete end-to-end execution sequence, from signal arrival through to fill completion and EQS scoring. The flow shows the happy path (signal → risk approval → dispatch → fill → score) and the three primary failure paths (risk rejection, broker rejection with retry, and timeout with retry).

02 End-to-End Execution Flow — Tick-Driven Async Pipeline From signal arrival → order submission → fill callback → EQS scoring

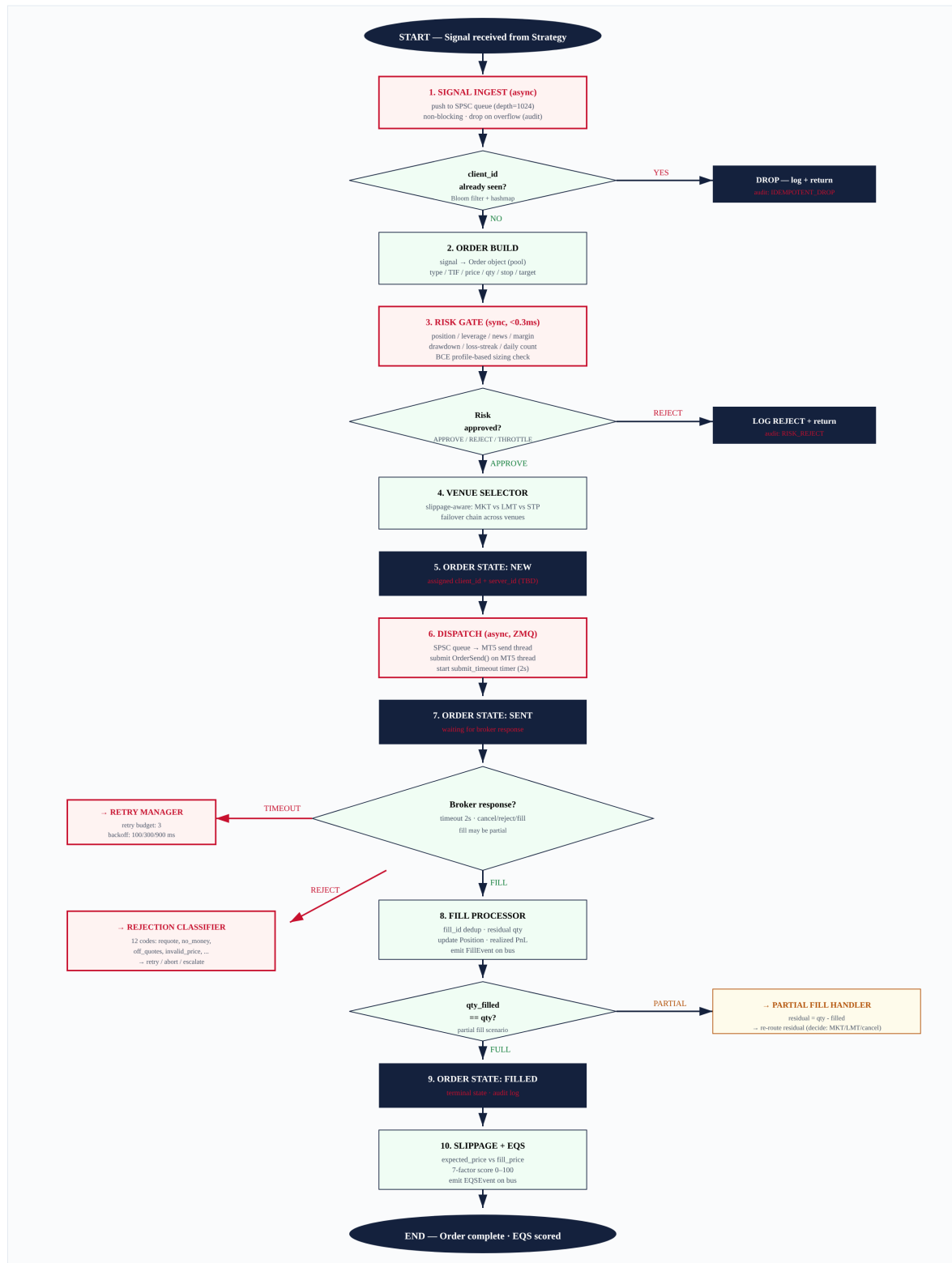


Figure 6.1 — End-to-end execution flow. Happy path: signal → risk → dispatch → fill → EQS. Three failure paths: risk reject, broker reject, submit timeout.

Happy Path Sequence

On the happy path, a signal arrives at the SignalIngestor and is pushed onto the hot-path ingress queue. The hot-path thread picks it up within microseconds, runs IdempotencyGuard (Bloom filter check), OrderBuilder (signal → Order object), RiskGateClient (synchronous call to RiskGate, returns APPROVE), VenueSelector (chooses MKT order for this signal), and transitions the order to VALIDATED. The OrderDispatcher then pushes the order onto the bridge-tx queue. The bridge-tx thread (CPU 6) picks up the order and calls MT5 OrderSend(), which is the slowest operation in the pipeline (0.5 ms p50, 1.5 ms p99). When MT5 returns a fill, the bridge-rx thread (CPU 6) receives the OnTrade callback and pushes the fill event onto the hot-path ingress queue. The hot-path thread's FillProcessor updates the order state to FILLED, updates the Position aggregate, and publishes a FillEvent on the ZMQ bus. The SlippageMonitor and EQS (on CPU 5) consume the FillEvent asynchronously to compute slippage and quality score.

Failure Path 1 — Risk Rejection

If the RiskGate returns REJECT (e.g., position size exceeds limit), the order is logged with the reject reason, audited, and the flow terminates. No broker call is made. The strategy coordinator is notified via the bus (so it can adjust future signals). This is the cheapest failure path — total latency is 0.45 ms (the risk gate call) and no broker resources are consumed.

Failure Path 2 — Broker Rejection with Retry

If MT5 returns a rejection (e.g., REQUOTE, retcode 10004), the RejectionClassifier categorizes it as RETRYABLE. The RetryManager schedules a retry with exponential backoff (100 ms for the first retry, 300 ms for the second, 900 ms for the third, plus ± 20 ms jitter). The order re-enters the NEW state with a new client_id (to avoid idempotency issues), and the flow restarts from OrderBuilder. After 3 retries, the order is aborted and the operator is notified via email. Total worst-case latency for a retried order is 1.36 seconds (3 retries with cumulative backoff), but the typical case is much faster (most retries succeed on the first attempt).

Failure Path 3 — Submit Timeout

If MT5 OrderSend does not return within 2 seconds, the order is marked as ERROR and passed to the ReconciliationEngine. The engine queries MT5 for the order state: if MT5 has the order, it is adopted into local state and the flow continues from fill processing; if MT5 does not have the order, it is retried (treated as a transient failure). This path handles the case where the MT5 terminal is slow or unresponsive but not disconnected. If the bridge is fully disconnected (detected via heartbeat), all SENT orders are marked as ERROR and the kill switch is engaged.

Chapter 7 — Order Lifecycle — State Machine

The order lifecycle is modeled as a finite state machine with 7 states and 14 transitions. The state machine is the single source of truth for order state; all components that need to know an order's state query the OrderManager, which owns the FSM. State transitions are atomic (protected by a per-order spinlock) and audited (every transition is logged with before-state, after-state, trigger, and timestamp).

03 Order Lifecycle — State Machine & Transition Reference

7 states · 14 transitions · 4 terminal states

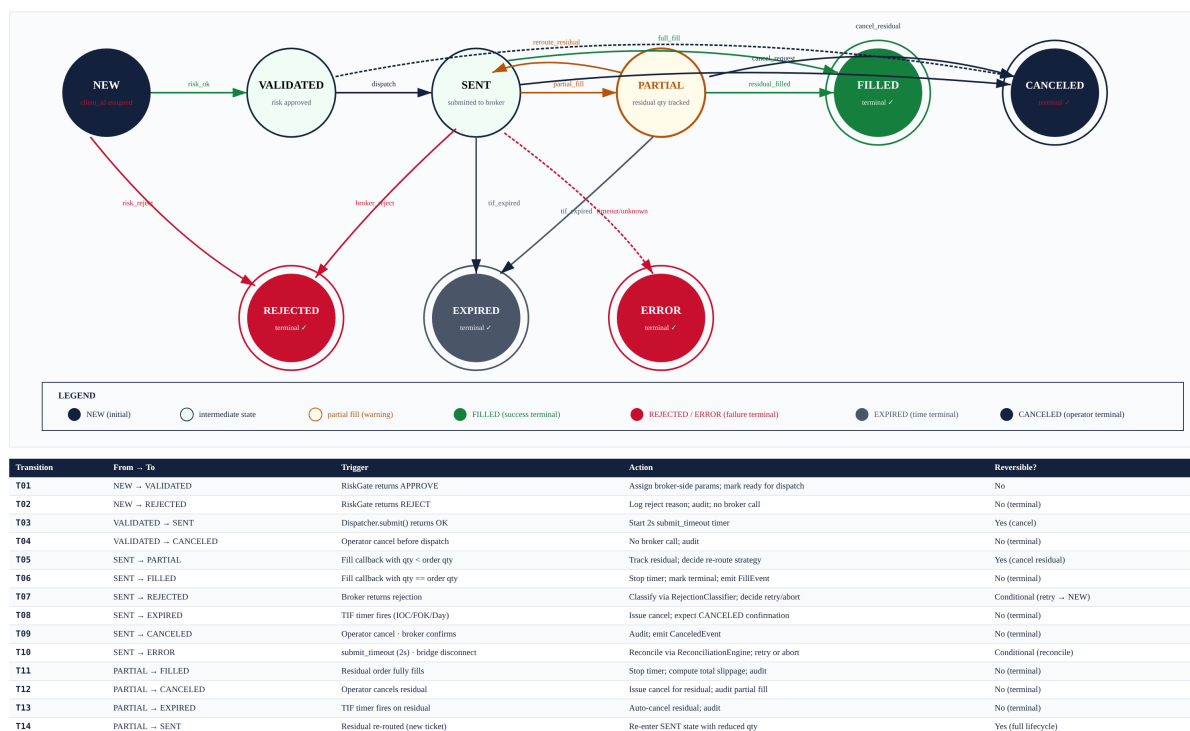


Figure 7.1 — Order lifecycle state machine. 7 states (NEW, VALIDATED, SENT, PARTIAL, FILLED, CANCELED, REJECTED, EXPIRED, ERROR) with 14 transitions. 4 terminal states are double-ringed.

State Definitions

NEW (initial)

The order has been received from the strategy and assigned a client_id, but has not yet been validated by the risk gate. This is the entry state for every order.

VALIDATED

The risk gate has approved the order. The order has been built (type, TIF, price, qty assigned) and is ready for dispatch to the broker. A VALIDATED order can be canceled before dispatch (transition T04).

SENT

The order has been submitted to the broker via OrderSend(). A 2-second submit_timeout timer is started; if the broker does not respond within 2 seconds, the order transitions to ERROR (transition T10). From SENT, the order can transition to PARTIAL (partial fill), FILLED (full fill), REJECTED (broker reject), EXPIRED (TIF

expired), CANCELED (operator cancel), or ERROR (timeout/disconnect).

PARTIAL

The broker has partially filled the order ($qty_filled < qty_requested$). The residual quantity is tracked, and the order waits for either additional fills ($\rightarrow$ FILLED via T11), operator cancel ($\rightarrow$ CANCELED via T12), TIF expiry ($\rightarrow$ EXPIRED via T13), or re-routing of the residual ($\rightarrow$ SENT via T14, with a new `client_id` for the residual portion).

FILLED (terminal, success)

The order has been fully filled. The fill is recorded, the Position is updated, and the order is moved to the audit log. This is the success terminal state.

CANCELED (terminal, operator)

The order has been canceled by operator action. Any partial fills are retained in the Position; the unfilled portion is abandoned. This is a terminal state.

REJECTED (terminal, failure)

The order was rejected either by the risk gate (transition T02, before broker submission) or by the broker (transition T07, after submission). The rejection reason is recorded. This is a terminal state — rejected orders are not retried directly; instead, the RetryManager creates a new order (with a new `client_id`) if the rejection is retryable.

EXPIRED (terminal, time)

The order's Time-In-Force (IOC, FOK, or DAY) has expired before the order was filled. Any partial fills are retained; the unfilled portion is abandoned. This is a terminal state.

ERROR (terminal, system)

A system error occurred (submit timeout, bridge disconnect, unknown broker response). The order is reconciled against broker state; if reconciliation succeeds, the order may be re-classified to FILLED or CANCELED. If reconciliation fails, the order remains in ERROR and the operator is paged. This is a terminal state pending reconciliation.

Transition Reference

The 14 transitions are documented in Figure 7.1. Each transition has a defined trigger, an action, and a reversibility property. Terminal transitions (to FILLED, CANCELED, REJECTED, EXPIRED, ERROR) are irreversible — once an order reaches a terminal state, it cannot leave. The only exception is ERROR, which can be re-classified to FILLED or CANCELED after successful reconciliation (this is not a state transition in the FSM sense, but a reclassification in the audit log).

FSM Invariants

The FSM enforces several invariants that are verified by property-based tests in CI:

- Every order starts in NEW and ends in exactly one terminal state (FILLED, CANCELED, REJECTED, EXPIRED, or ERROR).

- An order can visit PARTIAL at most once (partial fills accumulate within the state, not across visits).
- Once SENT, an order cannot return to NEW or VALIDATED (no "un-dispatch").
- A terminal state cannot be exited (transitions from terminal states are forbidden).
- Every transition is audited with before-state, after-state, trigger, and timestamp.
- The OrderManager is the single owner of order state; no other component may modify state directly.

Chapter 8 — Slippage Monitoring & Execution Quality Scoring

Slippage monitoring and execution quality scoring (EQS) are the IEE's quality feedback mechanisms. SlippageMonitor computes the difference between expected and realized fill prices on every fill, maintaining rolling statistics and flagging outliers. EQS produces a 0-100 score per order using a seven-factor weighted model, allowing operators and strategies to assess execution quality over time. Both run on the monitor thread (CPU 5), off the hot path, to avoid impacting latency.

04 Slippage Monitoring & Execution Quality Scoring (EQS)

Realized vs expected · 7-factor weighted score · outlier detection

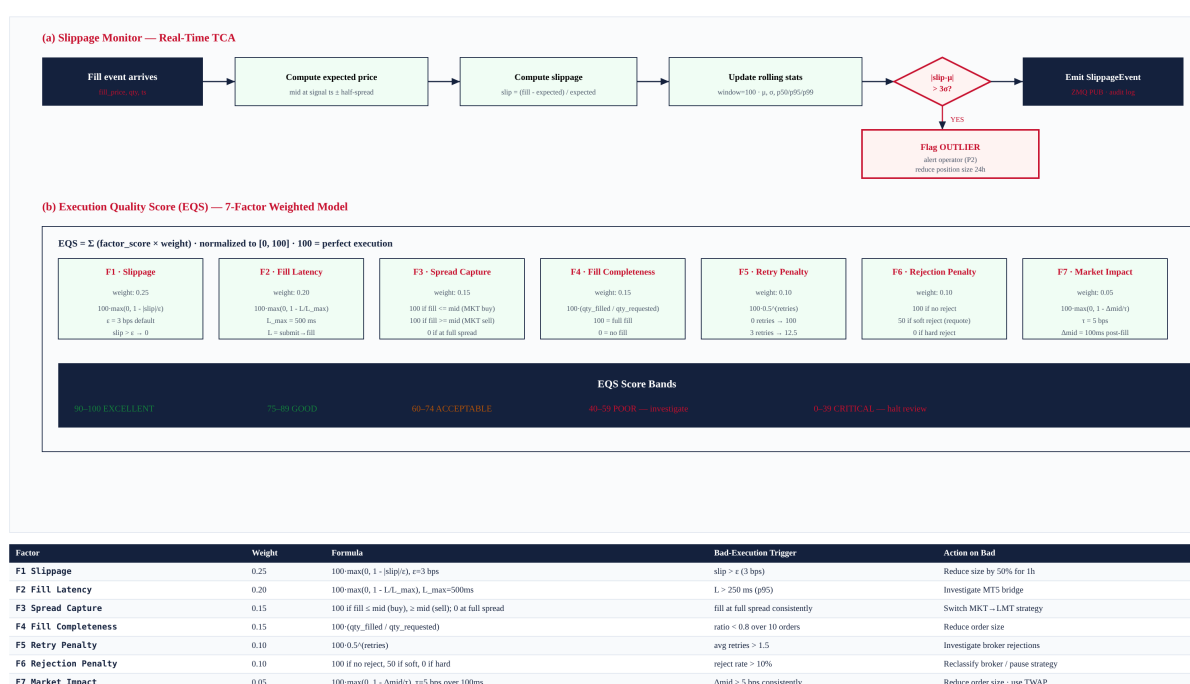


Figure 8.1 — (a) Slippage monitor pipeline; (b) EQS seven-factor weighted model with score bands.

Slippage Monitor

When a fill arrives, the SlippageMonitor computes the expected fill price as the mid price at signal time, adjusted by half the spread for marketable orders (a market buy is expected to fill at ask, a market sell at bid). The slippage is then computed as $(\text{fill_price} - \text{expected_price}) / \text{expected_price}$, expressed in basis points. The monitor maintains a rolling 100-trade window with mean (μ), standard deviation (σ), and percentiles (p50, p95, p99). If a fill's slippage exceeds $\mu + 3\sigma$, it is flagged as an outlier, triggering a P2 operator alert and a 1-hour position size reduction of 50%.

EQS — Seven-Factor Weighted Model

The Execution Quality Score is computed for every order that reaches a terminal state (FILLED, CANCELED, REJECTED, EXPIRED, ERROR). The score is a weighted sum of seven factors, each normalized to [0, 100], with weights summing to 1.0:

Factor	Weight	What It Measures	Bad-Execution Trigger
F1 Slippage	0.25	Absolute slippage vs expected price	slip > 3 bps
F2 Fill Latency	0.20	Time from submit to fill confirmation	L > 250 ms (p95)
F3 Spread Capture	0.15	How favorably the fill compares to mid	fill at full spread
F4 Fill Completeness	0.15	Ratio of filled qty to requested qty	ratio < 0.8
F5 Retry Penalty	0.10	Number of retries before success	avg retries > 1.5
F6 Rejection Penalty	0.10	Whether the order was rejected	reject rate > 10%
F7 Market Impact	0.05	Price movement 100ms after fill	Δ mid > 5 bps

EQS Score Bands

The composite EQS score is interpreted on five bands:

- 90-100: EXCELLENT — execution matched or beat expectations. No action.
- 75-89: GOOD — execution within normal parameters. No action.
- 60-74: ACCEPTABLE — execution slightly below expectations. Monitor.
- 40-59: POOR — execution significantly below expectations. Investigate root cause.
- 0-39: CRITICAL — execution unacceptable. Auto-pause strategy; engage risk-off mode.

EQS Feedback Loop

The EQS score feeds back into the system in three ways. First, the rolling 10-order average EQS is exposed as a Prometheus metric, allowing operators to monitor execution quality trends. Second, if the rolling average drops below 40 (CRITICAL), the IEE auto-pauses the strategy and engages risk-off mode, halting new orders until the operator intervenes. Third, the EQS score is published on the event bus, allowing the strategy coordinator to adjust its behavior — for example, reducing position size or switching from market to limit orders when execution quality is poor.

TCA — Transaction Cost Analysis

The TCACollector aggregates transaction cost data for the daily operator report. For each order, it records the spread cost (half-spread $\times$ qty), commission cost (commission rate $\times$ notional), and slippage cost (|fill - expected| $\times$ qty). The daily report aggregates these across all orders, broken down by symbol, strategy, and venue. The report is emailed to the operator and uploaded to the investor portal at 00:30 UTC daily.

Chapter 9 — Order Retry System

The RetryManager implements automatic retry of failed orders with exponential backoff and jitter. The retry budget is 3 per order — after 3 retries, the order is aborted and the operator is notified. The backoff schedule is 100ms / 300ms / 900ms (capped at 2000ms for any subsequent retry), plus ± 20 ms uniform jitter to avoid thundering-herd effects when many orders fail simultaneously.

Retry Strategy

The retry strategy is designed to handle transient failures (broker requotes, momentary price gaps, brief network hiccups) without overwhelming the broker. The exponential backoff gives the broker time to recover between retries, and the jitter ensures that if many orders fail at the same time (e.g., during a news event), they don't all retry at the same instant. The budget of 3 retries is calibrated to balance recovery probability against total latency: most transient failures recover on the first retry, and the third retry catches the long tail. Beyond 3 retries, the failure is likely persistent (e.g., broker is down) and operator intervention is required.

Backoff Schedule

Retry #	Base Delay	Jitter ($\pm$)	Total Delay	Cumulative	Action on Failure
1	100 ms	20 ms	80-120 ms	~100 ms	Schedule retry 2
2	300 ms	20 ms	280-320 ms	~420 ms	Schedule retry 3
3	900 ms	20 ms	880-920 ms	~1.34 s	ABORT · notify operator
4+ (capped)	2000 ms	20 ms	1980-2020 ms	N/A	(not reached; budget=3)

Retry Eligibility

Not all failures are retryable. The RejectionClassifier determines retry eligibility based on the MT5 return code:

- **RETRYABLE** (auto-retry): `REQUOTE` (10004), `PRICE_OFF` (10015), `NO_PRICES` (10019), `TOO_MANY_REQ` (10024). These are transient and likely to succeed on retry.
- **SOFT** (reduce size, then retry once): `PRICE_CHANGED` (10008), `INVALID_VOLUME` (10013), `NO_MONEY` (10014). These indicate the order was nearly valid; reducing size 50% and retrying often succeeds.
- **FATAL** (abort, do not retry): `MARKET_CLOSED` (10018), `BROKER_BLOCKED` (10026), `DISABLED`, `NO_CONNECTION`. These indicate the broker cannot accept any order; retrying is futile.
- **UNKNOWN** (abort): any unrecognized retcode. Conservative default — do not retry what we don't understand.

Retry Mechanics

When a retry is scheduled, the RetryManager creates a new Order object with a new `client_id` (to avoid idempotency conflicts) but otherwise identical parameters (except for **SOFT** retries, where the size is reduced

50%). The new order enters the NEW state and flows through the normal pipeline. The original order remains in its terminal state (REJECTED or ERROR) for audit purposes. The retry counter is tracked by client_id lineage: each retry knows which original order it descends from, allowing the RetryManager to enforce the budget of 3 across the retry chain.

Jitter Rationale

The $\pm 20\text{ms}$ uniform jitter is critical for systems that handle many simultaneous orders. Without jitter, if 100 orders fail at the same instant (e.g., during a brief broker hiccup), all 100 would retry at exactly 100ms, overwhelming the broker with a synchronized burst and likely causing another round of failures. With jitter, the retries spread over a 40ms window (80-120ms), smoothing the load on the broker. The 20ms jitter magnitude is calibrated to be small relative to the backoff interval (20% of the shortest interval) but large enough to provide meaningful spreading (100 retries spread over 40ms = 2.5 retries per millisecond, which is well within broker capacity).

Chapter 10 — Partial Fills Handling

Partial fills occur when the broker fills only part of an order's requested quantity. This is common for large orders in less liquid markets, and for limit orders that are matched against multiple counterparties. The IEE handles partial fills by transitioning the order to PARTIAL state, tracking the residual quantity, and applying a configurable residual policy (re-route, cancel, or wait).

Residual Quantity Tracking

When a partial fill arrives, the FillProcessor computes the residual quantity as $\text{qty_requested} - \text{qty_filled}$. The residual is tracked in the Order object and reflected in the Position aggregate (which now holds the partially-filled position). The order transitions to PARTIAL state, where it waits for one of four events: (1) additional fills, which reduce the residual further; (2) operator cancel, which abandons the residual; (3) TIF expiry, which auto-cancels the residual; or (4) re-routing, which creates a new order for the residual quantity.

Residual Policies

The strategy coordinator specifies a residual policy when placing the order. The IEE supports four policies:

- **RE_ROUTE_AS_MARKET**: The residual is converted to a market order and submitted immediately. Used when the strategy needs the position established quickly and is willing to pay the spread.
- **RE_ROUTE_AS_LIMIT**: The residual is submitted as a limit order at the original limit price. Used when the strategy can wait for a better fill.
- **CANCEL_RESIDUAL**: The residual is abandoned. Used when the partial fill is sufficient for the strategy's needs (e.g., the position is now large enough).
- **WAIT_FOR_FILL**: The residual remains in the market as the original order. Used for IOC orders that the broker has not yet fully matched.

Multiple Partial Fills

An order can receive multiple partial fills before reaching a terminal state. Each partial fill is processed independently: the FillProcessor updates the cumulative filled quantity, the residual, and the Position; emits a FillEvent; and updates the EQS factors (F4 Fill Completeness reflects the cumulative ratio). The order remains in PARTIAL state until the residual reaches zero (→ FILLED) or one of the other terminal transitions fires. Fill IDs are deduplicated via Bloom filter, preventing double-counting if the broker sends duplicate fill notifications.

Partial Fill Audit

Every partial fill is audited with the fill ID, fill quantity, fill price, cumulative filled quantity, residual quantity, and timestamp. The audit log allows operators to reconstruct the complete fill sequence for any order, which is essential for post-trade analysis and dispute resolution with the broker. The audit log is also used by the TCACollector to compute the volume-weighted average fill price (VWAP) for each order, which feeds into the EQS F1 Slippage factor.

Chapter 11 — Broker Rejection Handling

The RejectionClassifier maps MT5 return codes to 12 RejectCode values, each routed to one of three actions: RETRYABLE (auto-retry via RetryManager), SOFT (reduce size and retry once), or FATAL (abort and escalate). The classifier is the IEE's defense against the wide variety of rejection codes that MT5 brokers can return, translating them into a small number of well-defined actions.

MT5 Return Code Mapping

MT5 retcode	RejectCode	Meaning	Category	Action
10004	REQUOTE	No price for requested slip	RETRYABLE	retry (refresh price)
10006	REQUEST_CANCELED	Request cancelled by client	FATAL	abort
10008	PRICE_CHANGED	Price has changed since request	SOFT	reduce size 50% + retry
10013	INVALID_VOLUME	Volume not in step or range	SOFT	reduce size 50% + retry
10014	NO_MONEY	Insufficient margin	SOFT	reduce size 50% + retry
10015	PRICE_OFF	No quotes for symbol	RETRYABLE	retry (backoff 1s)
10016	PRICE_EXPIRED	Price expired before reach	RETRYABLE	retry
10018	MARKET_CLOSED	Market closed for symbol	FATAL	abort (queue for open)
10019	NO_PRICES	No prices for symbol	RETRYABLE	retry (backoff 1s)
10021	NO_QUOTES	No quotes for order type	RETRYABLE	retry
10024	TOO_MANY_REQUESTS	Rate limit exceeded	RETRYABLE	retry (backoff 2s)
10026	BROKER_BLOCKED	Broker blocked the order	FATAL	abort + kill switch
(other)	UNKNOWN	Unrecognized retcode	FATAL	abort (conservative)

FATAL Rejection Handling

FATAL rejections indicate that the broker cannot accept any order at this time. The order is aborted immediately, and the operator is paged via PagerDuty (P1 severity). For BROKER_BLOCKED (10026), the kill switch is engaged, halting all new orders and flattening existing positions. For MARKET_CLOSED (10018), the order is queued for re-submission when the market opens (if the strategy coordinator's policy allows). For

UNKNOWN retcodes, the order is aborted conservatively — we do not retry what we do not understand, and the operator is alerted to investigate.

Rejection Rate Monitoring

The IEE monitors the rejection rate per broker and per symbol, exposing it as a Prometheus metric. If the rejection rate exceeds 10% over a 5-minute window, a P2 alert is fired. If it exceeds 25%, a P1 alert is fired and the kill switch is engaged. High rejection rates typically indicate either a broker problem (e.g., the broker is throttling us due to excessive order submission) or a strategy problem (e.g., the strategy is submitting orders with stale prices). The monitoring allows operators to distinguish between these cases and take appropriate action.

Chapter 12 — Error Recovery — Reconciliation & State Repair

The IEE's error recovery subsystem handles failures that cannot be resolved by simple retry: orphan orders (local state has an order the broker doesn't), phantom orders (broker has an order local state doesn't), and stale state after a process restart or failover. The ReconciliationEngine runs every 5 seconds, comparing local state against broker state and resolving discrepancies according to defined policies.

05 Error Recovery — Retry, Rejection, Reconciliation

3-tier retry · 12 reject codes · orphan/phantom resolution

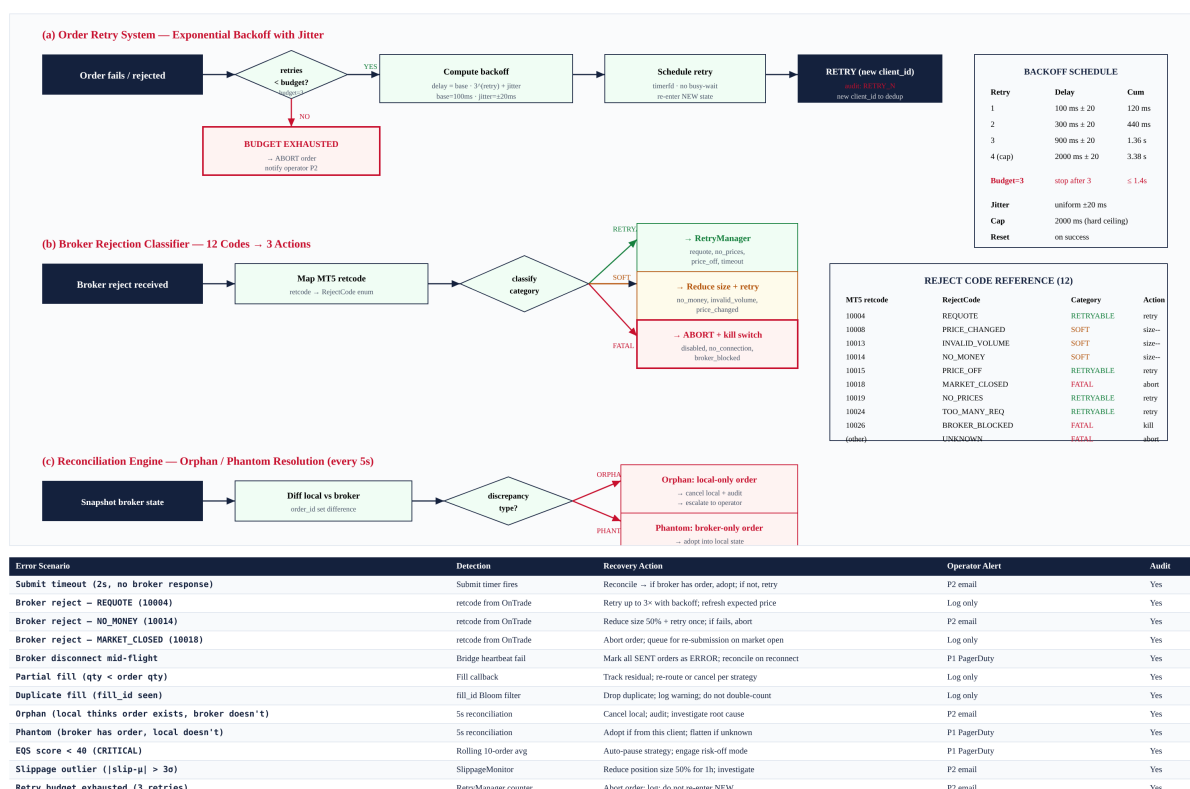


Figure 12.1 — Three error recovery subsystems: (a) retry manager with exponential backoff, (b) rejection classifier with 12-code mapping, (c) reconciliation engine for orphan/phantom resolution.

Reconciliation Engine

Every 5 seconds, the ReconciliationEngine queries the broker for all open orders and compares them against the local OrderManager state. The comparison produces three sets: (1) orders in both local and broker state (consistent — no action), (2) orders in local only (orphans — local thinks an order exists, broker doesn't), and (3) orders in broker only (phantoms — broker has an order local doesn't). Each discrepancy type has a defined resolution policy.

Orphan Resolution

An orphan order (local has, broker doesn't) typically indicates that the broker rejected or expired the order but the rejection notification was lost (e.g., due to a network hiccup during the callback). Resolution: cancel the local order, audit the discrepancy, and notify the operator (P2). If the order had partial fills, the Position is left intact

(the partial fill did happen); only the unfilled residual is canceled. Orphans are common after broker disconnects and are usually benign, but they must be resolved to prevent local state from drifting further from reality.

Phantom Resolution

A phantom order (broker has, local doesn't) is more serious — it means we have an order in the market that we don't know about. Resolution: if the order's `client_id` matches one of ours (we placed it but lost local state, e.g., due to a process crash), adopt it into local state and continue normal processing. If the `client_id` is not ours (someone else placed it, possibly an attacker or a stale session), flatten it immediately (submit a closing order) and page the operator (P1). Phantom orders with foreign `client_ids` are extremely rare and usually indicate a security incident.

State Recovery After Restart

When the IEE process restarts (after a crash, upgrade, or failover), it loads hot state from Redis (which is replicated from the primary to the standby VPS). The state includes all active orders, their current state, the Position aggregate, and rolling statistics. After loading, the `ReconciliationEngine` runs immediately (not waiting for the 5-second interval) to verify that the loaded state matches broker reality. Any discrepancies are resolved as described above. Total restart-to-ready time is under 3 seconds, dominated by Redis load (1s) and reconciliation (1-2s depending on order count).

Error Code Reference

The complete error code reference is shown in Figure 12.1. The 14 error scenarios cover every failure mode the IEE can encounter in production, from submit timeouts to idempotency violations to kill switch engagements. Each scenario has a defined detection mechanism, recovery action, operator alert level, and audit requirement. Operators should familiarize themselves with this table — it is the basis for all post-incident review and root-cause analysis.

Chapter 13 — Performance Benchmarks

The IEE is designed to meet specific, measurable performance targets. These targets are enforced as CI gates — a build that fails to meet them cannot be promoted to canary. This chapter documents the latency budget, throughput targets, resource envelope, and the engineering techniques used to achieve them.

06 Performance Benchmarks — Latency, Throughput, Resource Envelope

Tick-to-fill budget · 50k ticks/s sustained · <256MB RAM



Figure 13.1 — Signal-to-fill latency budget breakdown with per-stage p50/p99/p99.9, plus resource envelope (CPU, RAM, throughput, heap allocations).

Latency Budget

The internal signal-to-fill latency budget (excluding broker round-trip) is 2.0 ms at p99, achieved by a 0.95 ms p50 hot path. The end-to-end budget including broker round-trip is 6.0 ms at p99. The largest budget consumers are the RiskGate synchronous call (0.15 ms p50, 0.30 ms p99) and the MT5 OrderSend (0.50 ms p50, 1.50 ms p99). The broker round-trip is the largest single component (1.50 ms p50, 4.00 ms p99) but is not under our control. The full per-stage breakdown is shown in Figure 13.1.

Throughput Targets

Metric	Target	Sustained	Burst	Measurement
Tick ingestion	50,000 ticks/s	1 hour	100,000 ticks/s (1 min)	Locust load test
Order submission	200 orders/s	5 min	500 orders/s (30s)	Custom C++ bench
Fill processing	500 fills/s	5 min	1000 fills/s (30s)	Custom C++ bench
Cancel processing	200 cancels/s	5 min	500 cancels/s (30s)	Custom C++ bench
Reconciliation cycle	every 5s	continuous	every 2s (under load)	Timer observation
Audit log write	1000 events/s	1 hour	5000 events/s (1 min)	I/O benchmark

Resource Envelope

The IEE is designed to operate within a strict resource envelope, ensuring predictable performance and enabling capacity planning. The envelope is enforced via cgroups and monitored via Prometheus:

- **CPU:** 2 cores pinned (CPU 2-3 isolated, NO_HZ_FULL, rcu_nocbs). 4 additional cores for L1/L4/L5/L6 (CPUs 4-7).
- **RAM:** under 256 MB resident per process, 4 GB hugepages reserved for ring buffers (2MB pages, 2048 pages).
- **Disk I/O:** under 5 MB/s for audit log (batched 10ms flush), under 1 MB/s for Redis replication.
- **Network:** under 10 Mbps for ZMQ (tick ingress + event bus), under 1 Mbps for Redis (state replication).
- **Heap allocations on hot path:** 0 per order (enforced by static analyzer; Order pool is pre-allocated).
- **Syscalls on hot path:** 0 per order (enforced by static analyzer; all I/O delegated to worker threads).

Performance Engineering Techniques

CPU Pinning & Isolation

The hot-path thread is pinned to CPU 2-3 via systemd CPUAffinity. The kernel is instructed to never schedule other tasks there via isolcpus=2,3. NO_HZ_FULL eliminates timer ticks on those cores, and rcu_nocbs=2,3 offloads RCU callbacks. The result is fewer than 1 kernel preemption per second under load, compared to thousands per second on a default kernel. This is the single most impactful change for latency predictability.

Lock-Free SPSC Queues

Inter-thread communication uses lock-free SPSC queues based on moodycamel::ConcurrentQueue. Each push and pop is a single atomic load and store, with no locks, no condition variables, and no syscalls. Cache-line padding (64 bytes) prevents false sharing between producer and consumer. The queues are sized for worst-case burst scenarios and apply drop-and-audit backpressure when full.

Pre-Allocation & Pool Allocation

All hot-path memory is pre-allocated at startup. The Order pool (4096 orders) is allocated as a contiguous array and managed via a free-list; allocation is a single pointer swap. Message buffers come from a slab allocator with size-class buckets. Ring buffers are sized at startup based on configured capacity. No malloc, no new, no std::vector resize, no std::string concatenation on the hot path. This gives O(1) allocation latency and a hard cap on memory usage.

Branchless Hot Path

The hot path is written to be branchless where possible, using ternary operators and arithmetic instead of if-statements. Branch mispredictions cost 10-20 nanoseconds each on modern CPUs, and unpredictable branches (e.g., "if order is rejected") can mispredict 50% of the time. The static analyzer flags any branch in the hot path that is not marked as __builtin_expect, forcing developers to either eliminate the branch or annotate it with predicted direction.

Chapter 14 — Validation Tests

The IEE is covered by a five-layer test pyramid: unit tests (per-component logic), integration tests (Pact contracts between layers), lifecycle tests (FSM transition coverage), performance tests (latency and throughput enforcement), and chaos tests (fault injection). The complete pyramid and per-subsystem coverage matrix are shown in Figure 14.1.

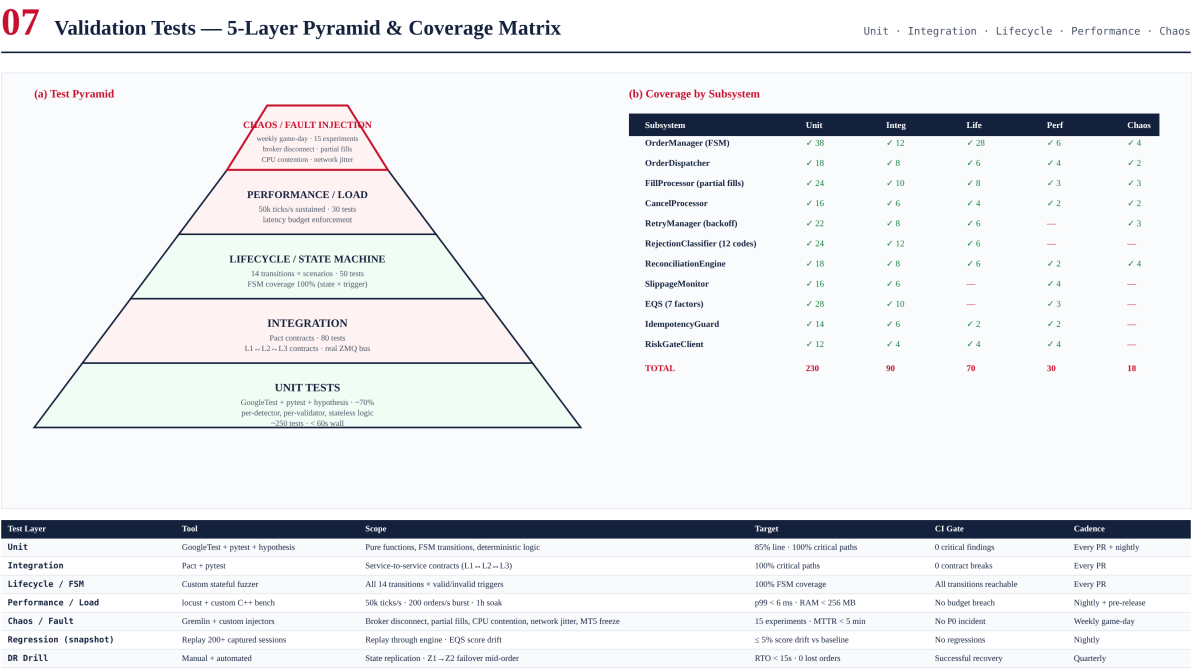


Figure 14.1 — Test pyramid (5 layers) with per-subsystem coverage matrix, plus test layer reference table.

Unit Test Cases — Sample

Unit tests cover pure functions and isolated components with all dependencies mocked. Property-based tests (via hypothesis) verify FSM invariants. Below are sample test cases for the OrderManager FSM and the RetryManager.

Sample Unit Tests — OrderManager FSM

Test ID	Scenario	Expected Transition	Severity
UT-FSM-001	NEW + risk_ok	NEW → VALIDATED	PASS
UT-FSM-002	NEW + risk_reject	NEW → REJECTED (terminal)	PASS
UT-FSM-003	VALIDATED + dispatch_ok	VALIDATED → SENT	PASS
UT-FSM-004	SENT + full_fill	SENT → FILLED (terminal)	PASS

Test ID	Scenario	Expected Transition	Severity
UT-FSM-005	SENT + partial_fill	SENT → PARTIAL	PASS
UT-FSM-006	PARTIAL + residual_fill	PARTIAL → FILLED (terminal)	PASS
UT-FSM-007	SENT + broker_reject (10004)	SENT → REJECTED (terminal)	PASS
UT-FSM-008	SENT + tif_expired	SENT → EXPIRED (terminal)	PASS
UT-FSM-009	SENT + submit_timeout (2s)	SENT → ERROR	PASS
UT-FSM-010	FILLED + any trigger	FATAL: terminal state cannot transition	FAIL-FATAL
UT-FSM-011	NEW + cancel_request	NEW → CANCELED (terminal)	PASS
UT-FSM-012	PARTIAL + reroute_residual	PARTIAL → SENT (new client_id)	PASS

Sample Unit Tests — RetryManager

Test ID	Scenario	Expected Behavior	Severity
UT-RTY-001	REQUOTE + retry 0	schedule retry @ 100ms ± 20ms	PASS
UT-RTY-002	REQUOTE + retry 1	schedule retry @ 300ms ± 20ms	PASS
UT-RTY-003	REQUOTE + retry 2	schedule retry @ 900ms ± 20ms	PASS
UT-RTY-004	REQUOTE + retry 3	ABORT + P2 alert	PASS
UT-RTY-005	NO_MONEY + retry 0	reduce size 50% + retry @ 100ms	PASS
UT-RTY-006	MARKET_CLOSED + retry 0	ABORT (FATAL, no retry)	PASS
UT-RTY-007	BROKER_BLOCKED + retry 0	ABORT + kill switch (FATAL)	PASS
UT-RTY-008	Jitter distribution	1000 samples uniform in [80, 120] ms	PASS
UT-RTY-009	Budget reset on success	retry counter resets to 0 after FILLED	PASS
UT-RTY-010	New client_id per retry	each retry gets unique client_id	PASS

Performance Test Cases

Performance tests enforce the latency budget and throughput targets. They run nightly and before every release. A performance regression (e.g., p99 latency increases by 20%) blocks the release until investigated and resolved.

Test ID	Scenario	Target	Gate
PT-001	Tick ingestion sustained 1h	50,000 ticks/s · 0 drops	Must meet
PT-002	Order submission burst 30s	500 orders/s · 0 rejects (except retryable)	Must meet
PT-003	Signal-to-dispatch p99	< 0.50 ms (excludes MT5 send)	Must meet
PT-004	Signal-to-fill p99 (internal)	< 2.00 ms (excludes broker RTT)	Must meet
PT-005	Signal-to-fill p99 (end-to-end)	< 6.00 ms (includes broker RTT)	Must meet
PT-006	RAM resident set after 1h load	< 256 MB	Must meet
PT-007	Heap allocations per order (hot path)	0 (verified by LD_PRELOAD allocator)	Must meet
PT-008	CPU utilization on CPU 2-3 (hot path)	95-100% (busy-spin expected)	Must meet
PT-009	Reconciliation cycle under 1000 orders	< 50 ms	Must meet
PT-010	Restart-to-ready (state load + reconcile)	< 3.0 s	Must meet

Chaos Test Cases

Chaos tests inject realistic failures into a production-like staging environment. They run weekly and after major changes. Each experiment produces a postmortem regardless of outcome; experiments that reveal bugs feed back into unit tests.

Test ID	Experiment	Expected Behavior	Success Criteria
CT-001	Broker disconnect mid-flight	Mark SENT orders ERROR · page operator · reconcile on reconnect	0 lost orders · RTO < 15s
CT-002	MT5 terminal freeze (5s)	Submit timeouts · retry budget respected · no orphan orders	All orders terminal within 30s
CT-003	Partial fill storm (10 consecutive)	Residual tracked · policy applied · EQS reflects correctly	Cumulative qty matches
CT-004	Rejection storm (100 REQUOTE in 1s)	Retry budget respected · no thundering herd · backoff with jitter	Peak retry rate < 50/s
CT-005	CPU contention on hot-path cores	Latency p99 increases but no missed fills · alert fires	p99 < 5ms under contention
CT-006	Network jitter (50ms ± 20ms)	Fill callbacks delayed but processed · no duplicate fills	0 duplicate fills
CT-007	Redis (state cache) unavailable	Fallback to in-memory state · reconcile on reconnect	0 lost orders

Test ID	Experiment	Expected Behavior	Success Criteria
CT-008	Audit log disk full	Buffer in memory · alert · continue trading (audit buffered)	No trading impact
CT-009	Kill switch mid-order	Pending orders canceled · filled portion retained · halt	< 500ms end-to-end
CT-010	Process crash mid-fill	Restart · load state from Redis · reconcile · resume	0 lost fills

Test Coverage Summary

Test Layer	Count	Coverage Target	CI Gate	Cadence
Unit (per-component logic)	230	85% line · 100% critical paths	0 critical findings	Every PR + nightly
Integration (Pact contracts)	90	100% L1↔L2↔L3 contracts	0 contract breaks	Every PR
Lifecycle (FSM transitions)	70	100% of 14 transitions	All reachable	Every PR
Performance / Load	30	Latency + throughput + RAM	No budget breach	Nightly + pre-release
Chaos / Fault injection	18	15 experiments (3 extra)	No P0 incident	Weekly game-day
Regression (snapshot)	200+	Replay 200+ captured sessions	≤ 5% EQS drift	Nightly
DR Drill (Z1→Z2 mid-order)	1	0 lost orders · RTO < 15s	Successful recovery	Quarterly
Total	640+	—	—	—

Chapter 15 — Appendix A — Order State Machine Reference

This appendix provides a complete reference for the order state machine, including all states, transitions, and the rules governing transitions. It is intended as a quick lookup for engineers debugging order state issues.

A.1 State Reference

State	Type	Description	Audit Fields
NEW	initial	Signal received, awaiting risk validation	client_id, signal_id, received_at
VALIDATED	intermediate	Risk approved, ready for dispatch	risk_decision, validated_at, order_params
SENT	intermediate	Submitted to broker, awaiting response	broker_ticket, submitted_at, submit_timeout
PARTIAL	intermediate	Partially filled, residual tracked	filled_qty, residual_qty, last_fill_id
FILLED	terminal (success)	Fully filled	fill_id, fill_price, fill_qty, filled_at
CANCELED	terminal (operator)	Canceled by operator	canceled_at, cancel_reason, partial_fill_qty
REJECTED	terminal (failure)	Rejected by risk or broker	reject_code, reject_reason, rejected_at
EXPIRED	terminal (time)	TIF expired	expired_at, tif_type, partial_fill_qty
ERROR	terminal (system)	System error (timeout, disconnect)	error_code, error_context, error_at

A.2 Transition Reference

The 14 transitions are documented in Figure 7.1. Each transition is atomic and audited. The FSM enforces the following rules:

- Every order enters via NEW and exits via exactly one terminal state.
- Terminal states (FILLED, CANCELED, REJECTED, EXPIRED, ERROR) cannot be exited.
- PARTIAL can transition to FILLED, CANCELED, EXPIRED, or back to SENT (via re-route).
- SENT can transition to PARTIAL, FILLED, REJECTED, EXPIRED, CANCELED, or ERROR.
- Every transition is logged with before-state, after-state, trigger, and timestamp.

Chapter 16 — Appendix B — Sample Execution Logs

This appendix shows the audit log entries for three representative execution scenarios: a successful market order, a partial fill with residual re-route, and a broker rejection with retry. The logs are shown in JSON form for readability; in production, they are serialized as FlatBuffers for performance.

B.1 Successful Market Order

```
{
  "events": [
    {
      "ts": 1718798400000000000,
      "type": "SIGNAL_RECEIVED",
      "client_id": "CLT-7F4A92-001",
      "signal_id": "SIG-MOMENTUM-2026-06-19-001",
      "symbol": "XAUUSD",
      "side": "BUY",
      "qty": 0.50,
      "order_type": "MARKET"
    },
    {
      "ts": 1718798400000150000,
      "type": "RISK_APPROVED",
      "client_id": "CLT-7F4A92-001",
      "decision": "APPROVE",
      "checks_passed": ["position", "leverage", "news", "margin", "drawdown"]
    },
    {
      "ts": 1718798400000280000,
      "type": "ORDER_DISPATCHED",
      "client_id": "CLT-7F4A92-001",
      "broker_ticket": 123456789,
      "order_type": "MARKET",
      "qty": 0.50,
      "price_at_dispatch": 1950.45
    },
    {
      "ts": 1718798400005800000,
      "type": "FILL_RECEIVED",
      "client_id": "CLT-7F4A92-001",
      "fill_id": "FILL-001",
      "fill_price": 1950.48,
      "fill_qty": 0.50,
      "commission": 3.41
    },
    {
      "ts": 1718798400005850000,
      "type": "ORDER_FILLED",
      "client_id": "CLT-7F4A92-001",
      "final_state": "FILLED",
      "total_filled_qty": 0.50,
      "vwap": 1950.48
    },
    {
      "ts": 1718798400005900000,
      "type": "EQS_SCORED",

```

```

"client_id": "CLT-7F4A92-001",
"score": 87.5,
"factors": {
  "F1_slippage": 92.0,
  "F2_latency": 95.0,
  "F3_spread_capture": 80.0,
  "F4_completeness": 100.0,
  "F5_retry_penalty": 100.0,
  "F6_rejection_penalty": 100.0,
  "F7_market_impact": 60.0
}
},
"summary": {
  "client_id": "CLT-7F4A92-001",
  "total_latency_ns": 5900000,
  "internal_latency_ns": 900000,
  "broker_rtt_ns": 5000000,
  "eqs_score": 87.5,
  "final_state": "FILLED"
}
}

```

B.2 Partial Fill with Residual Re-Route

```

{
  "events": [
    { "ts": 0, "type": "SIGNAL_RECEIVED", "client_id": "CLT-PARTIAL-001", "qty": 1.00, "order_type": "LIMIT", "price": 1950.00 },
    { "ts": 150000, "type": "RISK_APPROVED", "client_id": "CLT-PARTIAL-001" },
    { "ts": 280000, "type": "ORDER_DISPATCHED", "client_id": "CLT-PARTIAL-001", "broker_ticket": 222111333 },
    { "ts": 1800000, "type": "PARTIAL_FILL", "client_id": "CLT-PARTIAL-001", "fill_id": "FILL-P1", "fill_qty": 0.40, "fill_price": 1950.02, "residual": 0.60 },
    { "ts": 1850000, "type": "RESIDUAL_RE_ROUTE", "client_id": "CLT-PARTIAL-001", "residual_client_id": "CLT-PARTIAL-001-R1", "policy": "RE_ROUTE_AS_MARKET" },
    { "ts": 2000000, "type": "ORDER_DISPATCHED", "client_id": "CLT-PARTIAL-001-R1", "qty": 0.60, "order_type": "MARKET" },
    { "ts": 2500000, "type": "FILL_RECEIVED", "client_id": "CLT-PARTIAL-001-R1", "fill_id": "FILL-P2", "fill_price": 1950.50, "fill_qty": 0.60 },
    { "ts": 2510000, "type": "ORDER_FILLED", "client_id": "CLT-PARTIAL-001-R1", "final_state": "FILLED" },
    { "ts": 2520000, "type": "ORDER_FILLED", "client_id": "CLT-PARTIAL-001", "final_state": "FILLED", "total_filled_qty": 1.00, "vwap": 1950.308 }
  ]
}

```

B.3 Broker Rejection with Retry

```

{
  "events": [
    { "ts": 0, "type": "SIGNAL_RECEIVED", "client_id": "CLT-RETRY-001", "qty": 0.30, "order_type": "MARKET" },
    { "ts": 150000, "type": "RISK_APPROVED", "client_id": "CLT-RETRY-001" },
    { "ts": 280000, "type": "ORDER_DISPATCHED", "client_id": "CLT-RETRY-001",

```

```

"broker_ticket": 333222111 },
{ "ts": 780000, "type": "BROKER_REJECT", "client_id": "CLT-RETRY-001", "retcode":
10004, "reject_code": "REQUOTE", "category": "RETRYABLE" },
{ "ts": 790000, "type": "RETRY_SCHEDULED", "original_client_id": "CLT-RETRY-001",
"retry_client_id": "CLT-RETRY-001-R1", "retry_num": 1, "delay_ms": 110 },
{ "ts": 900000, "type": "ORDER_DISPATCHED", "client_id": "CLT-RETRY-001-R1",
"broker_ticket": 333222222, "price_at_dispatch": 1950.52 },
{ "ts": 1400000, "type": "FILL_RECEIVED", "client_id": "CLT-RETRY-001-R1",
"fill_id": "FILL-RT1", "fill_price": 1950.55, "fill_qty": 0.30 },
{ "ts": 1410000, "type": "ORDER_FILLED", "client_id": "CLT-RETRY-001-R1",
"final_state": "FILLED" },
{ "ts": 1410000, "type": "ORDER_REJECTED", "client_id": "CLT-RETRY-001",
"final_state": "REJECTED", "reason": "REQUOTE (retried as CLT-RETRY-001-R1)" }
],
"summary": {
"original_client_id": "CLT-RETRY-001",
"retries": 1,
"total_latency_ns": 1410000,
"final_state": "FILLED (via retry)",
"eqs_score": 72.5,
"factors": { "F5_retry_penalty": 50.0, "F1_slippage": 75.0 }
}
}

```

These three examples illustrate the IEE's behavior across the most common execution scenarios. The audit log is the authoritative record of every order's lifecycle and is used for post-trade analysis, dispute resolution with the broker, and regulatory compliance reporting.